

E4 – EXAMEN DE EVALUACIÓN

Curso	IFCD0182 – Desarrollo Web con Java		
Módulo Formativo	Módulo 1 2 3 4 Examen Final	Duración	120 minutos
Unidad Formativa	UF Spring Boot: REST, JPA y Spring Cloud	Fecha	29/05/2026
Docente	Doninger Ramón Casanova Casanova	DNI	79197448F
Alumno/a		DNI	

Instrucciones generales

Esta prueba práctica tiene una puntuación máxima de **20 puntos**. El objetivo es crear un microservicio Spring Boot con dos entidades (Curso y Alumno), registrarlo en Eureka y acceder a él a través de un API Gateway.

- Crea los tres proyectos en el orden indicado.
- La base de datos MariaDB debe estar encendida.
- Puedes consultar tus apuntes. Puedes usar la IA solo para entender errores concretos o recordar sintaxis.

Orden obligatorio de arranque

1. MariaDB encendida.
2. eureka-server en el puerto 8761.
3. formacion-service en el puerto 8081.
4. api-gateway en el puerto 8080.

¡¡MUCHA SUERTE!!

Enunciado de la práctica

Actividad: crea un microservicio REST para un centro de formación. El servicio gestionará cursos y alumnos dentro de la misma aplicación Spring Boot. Solo habrá un microservicio de negocio.

Estructura final esperada

```
centro-formacion/  
├── eureka-server/      → puerto 8761  
├── formacion-service/  → puerto 8081 | BD: formacion_db  
└── api-gateway/       → puerto 8080
```

Entidades

Entidad	Campo	Descripción
Curso	id (Long)	Clave primaria. Se genera automáticamente.
	titulo (String)	Nombre del curso.
	modalidad (String)	Valor: PRESENCIAL u ONLINE.
	duracion (int)	Duración en horas.
Alumno	id (Long)	Clave primaria. Se genera automáticamente.
	nombre (String)	Nombre completo del alumno.
	email (String)	Correo electrónico del alumno.
	cursold (Long)	Id del curso en el que está matriculado.

Base de datos

```
CREATE DATABASE IF NOT EXISTS formacion_db  
CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

Parte 1. eureka-server

Crea un proyecto Spring Boot con la dependencia Eureka Server y copia esta configuración:

```
# application.properties  
spring.application.name=eureka-server  
server.port=8761  
eureka.client.register-with-eureka=false  
eureka.client.fetch-registry=false
```

1. Anota la clase principal con `@EnableEurekaServer`.
2. Arranca el proyecto y abre `http://localhost:8761` en el navegador para confirmar que funciona.

Parte 2. formacion-service

Crea el microservicio principal. Dependencias necesarias: Spring Web, Spring Data JPA, MariaDB Driver y Eureka Discovery Client.

2.1 application.properties

```
spring.application.name=formacion-service
server.port=8081
spring.datasource.url=jdbc:mariadb://localhost:3306/formacion_db
spring.datasource.username=TU_USUARIO
spring.datasource.password=TU_PASSWORD
spring.datasource.driver-class-name=org.mariadb.jdbc.Driver
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
```

2.2 Entidades

Crea las dos clases en el paquete com.formacion.model. Las dos siguen la misma estructura:

1. Crea la clase Curso con los campos indicados en la tabla de entidades.
2. Anota la clase con `@Entity`. Anota el campo id con `@Id` y `@GeneratedValue`.
3. Genera los getters, setters y un constructor vacío (botón derecho → Generate en tu IDE).
4. Repite los tres pasos anteriores para la clase Alumno.

2.3 Repositorios

Crea dos interfaces en el paquete com.formacion.repository:

```
public interface CursoRepository extends JpaRepository<Curso, Long> { }

public interface AlumnoRepository extends JpaRepository<Alumno, Long> { }
```

2.4 Controladores REST

Crea CursoController en el paquete com.formacion.controller y completa los TODOs:

```
@RestController
@RequestMapping("/api/v1/cursos")
public class CursoController {

    private final CursoRepository repo;

    public CursoController(CursoRepository repo) { this.repo = repo; }

    // TODO 1: GET /api/v1/cursos
    // Devuelve ResponseEntity.ok(repo.findAll())

    // TODO 2: POST /api/v1/cursos
    // Recibe un Curso con @RequestBody
    // Devuelve ResponseEntity.ok(repo.save(curso))
}
```

Crea AlumnoController en el mismo paquete y completa los TODOs:

```
@RestController
@RequestMapping("/api/v1/alumnos")
public class AlumnoController {

    private final AlumnoRepository repo;

    public AlumnoController(AlumnoRepository repo) { this.repo = repo; }

    // TODO 3: GET /api/v1/alumnos
    // Devuelve ResponseEntity.ok(repo.findAll())

    // TODO 4: POST /api/v1/alumnos
    // Recibe un Alumno con @RequestBody
    // Devuelve ResponseEntity.ok(repo.save(alumno))
}
```

Parte 3. api-gateway

Crea un proyecto Spring Boot con la dependencia Spring Cloud Gateway y copia esta configuración tal cual:

```
# application.yml
spring:
  application:
    name: api-gateway
  cloud:
    gateway:
      routes:
        - id: formacion-service
          uri: lb://formacion-service
          predicates:
            - Path=/api/v1/**
  server:
    port: 8080
  eureka:
    client:
      service-url:
        defaultZone: http://localhost:8761/eureka/
```

1. No modifiques nada de este archivo. El Gateway enrutará todas las llamadas al servicio.

Pruebas obligatorias

Realiza las pruebas con Postman o el navegador y guarda una captura de cada una:

#	URL / Acción	Resultado esperado
1	http://localhost:8761	Panel Eureka visible. formacion-service aparece registrado.
2	POST http://localhost:8080/api/v1/cursos	Body: { "titulo": "Java Básico", "modalidad": "PRESENCIAL", "duracion": 30 }
3	GET http://localhost:8080/api/v1/cursos	Lista de cursos en JSON devuelta por el Gateway.

4	POST http://localhost:8080/api/v1/alumnos	Body: { "nombre": "Carlos García", "email": "carlos@email.com", "cursold": 1 }
5	GET http://localhost:8080/api/v1/alumnos	Lista de alumnos en JSON devuelta por el Gateway.

Pistas permitidas

- @Entity marca la clase como tabla en la base de datos.
- @Id + @GeneratedValue(strategy = GenerationType.IDENTITY) generan el id automáticamente.
- @RestController + @RequestMapping definen el controlador REST.
- @GetMapping para listar. @PostMapping para crear. @RequestBody para leer el JSON del cuerpo.
- ResponseEntity.ok(...) devuelve 200 con el contenido. repo.findAll() devuelve la lista. repo.save(...) guarda y devuelve el objeto guardado.

Errores frecuentes

Error	Causa probable	Qué revisar
No conecta a MariaDB	URL, usuario o contraseña incorrectos	Revisar application.properties y que MariaDB está encendida.
No se crean las tablas	Falta ddl-auto o dependencia JPA	Añadir spring.jpa.hibernate.ddl-auto=update.
Servicio no aparece en Eureka	Falta Eureka Discovery Client	Añadir la dependencia y revisar defaultZone.
Gateway devuelve 503	El servicio no está registrado aún	Esperar unos segundos. Comprobar el panel Eureka.
GET /{id} devuelve error 500	Optional mal usado	Usar .isPresent() antes de devolver el objeto.

Criterio de logro

La práctica se considerará superada si formacion-service aparece en Eureka, las tablas curso y alumno se crean automáticamente en MariaDB y los cuatro endpoints responden correctamente a través del API Gateway.

Lista de cotejo / rúbrica de corrección

Criterio	Puntuación máx.	Sí / No	Observaciones
eureka-server arranca correctamente y muestra el panel en http://localhost:8761.	4 puntos	☐ Sí ☐ No	
Las entidades Curso y Alumno están correctamente anotadas y las tablas se crean en MariaDB.	4 puntos	☐ Sí ☐ No	

Criterio	Puntuación máx.	Sí / No	Observaciones
Los cuatro TODOs del controlador están implementados (GET y POST de cursos; GET y POST de alumnos).	8 puntos	☐ Sí ☐ No	
api-gateway enruta las peticiones y las cinco pruebas funcionan a través del puerto 8080.	4 puntos	☐ Sí ☐ No	

Entrega

- Los tres proyectos comprimidos en un único archivo .zip.
- Captura del panel Eureka con formacion-service registrado.
- Captura del POST de cursos con la respuesta JSON.
- Captura del GET de cursos a través del Gateway.
- Captura del POST y GET de alumnos a través del Gateway.